I. GİRİŞ

Sistem güvenliği ve veri bütünlüğünün korunmasında, kritik klasörlerin ve yapılandırma dosyalarının izlenmesi öncelikli adımlardan biridir. Zararlı yazılımların sisteme dosya enjekte etmesi veya mevcut dosyaları manipüle etmesi gibi senaryoların anında tespit edilmesi gerekir.

Bu projenin amacı; arka planda minimum kaynak tüketerek çalışan, kullanıcı tarafından belirlenen hedef bir yerel dizini sürekli tarayan ve dosya düzeyindeki değişiklikleri (yeni dosya oluşturulması veya mevcut dosyanın değiştirilmesi) anlık olarak yakalayıp güvenlik logu üreten bir yazılım prototipi ortaya koymaktır. Projenin mimarisi, ilerleyen süreçlerde e-posta bildirimi veya veri tabanı kaydı gibi yeni modüllerin kolayca eklenebilmesine imkan tanıyacak esneklikte tasarlanmıştır.

II. YAZILIM MİMARİSİ VE TASARIM MODELİ

Uygulamanın nesne yönelimli yapısı oluşturulurken, sınıfların birbirine sıkı sıkıya bağlı olmamasını (loose coupling) sağlamak adına **Observer Tasarım Deseni** temel alınmıştır.

A. Sınıf Yapıları ve Rolleri

- **`IObserver` (Arayüz):** Bildirim mekanizmasının temelini oluşturan soyut sınıftır. İçerisinde saf sanal (`pure virtual`) olarak tanımlanmış `onNotify(const std::string& message)` fonksiyonunu barındırır. Bu sayede sistemi dinleyecek yeni bir yapı ekleneceğinde sadece bu sınıftan türetilmesi yeterli olmaktadır.
- **`SecurityLogger` (Gözlemci sınıfı):** `IObserver` arayüzünden kamuya açık (`public`) olarak türetilmiştir. Tetiklenen dosya olaylarını devralarak konsol ekranına `[GUVENLIK UYARISI]` başlığıyla anlamlı loglar basar.
- **`FileMonitor` (İzleyici ana sınıf):** Klasörün takibinden sorumlu merkez sınıftır. İzlenecek dizinin yolunu saklar. İçerisindeki `std::vector<IObserver*>` listesi sayesinde, kendisini dinleyen tüm gözlemcileri dinamik olarak kayıt altında tutar (`addObserver`).

B. OOP Prensiplerinin Uygulanışı

Sistemde çok biçimlilik (Polymorphism) aktif olarak kullanılmıştır. `FileMonitor` sınıfı, kendisini dinleyen nesnenin `SecurityLogger` mı yoksa başka bir sınıf mı olduğunu bilmez; sadece `IObserver` işaretçisi (`pointer`) üzerinden bildirim fonksiyonunu çağırır. Bu durum kodun soyutlama gücünü artırmış ve bağımlılıkları minimize etmiştir.

III. SİSTEMİN GERÇEKLENMESİ VE ÇALIŞMA AKIŞI

Uygulamanın yapısal tanımlamaları `Monitor.h` başlık dosyasında, ana mantık ve döngü akışı ise `main.cpp` dosyasında kodlanmıştır. Sistemin çalışma algoritması şu şekildedir:

1. **İlk Durum Analizi:** `FileMonitor::startMonitoring()` fonksiyonu çağrıldığında, C++17 ile gelen `std::filesystem` kütüphanesi kullanılarak hedef klasörün içerisindeki mevcut tüm dosyalar taranır. Her dosyanın tam yolu ve son değiştirilme tarihi (`fs::last_write_time`), anahtar-değer çifti olarak bir `std::map<std::string, fs::file_time_type>` yapısına kaydedilir. Bu işlem uygulamanın referans noktasını oluşturur.
2. **Gerçek Zamanlı Tarama (Döngü):** Uygulama `while(true)` döngüsüyle sürekli aktif kalır. İşlemciyi gereksiz yere yormamak adına `std::this_thread::sleep_for` fonksiyonu kullanılarak döngü her saniyede bir (1000 milisaniye) tetiklenecek şekilde uyutulur.
3. **Değişiklik Algılama Mantığı:** Her saniye klasör `fs::recursive_directory_iterator` ile baştan aşağı taranır ve o anki durum `map` veritabanı ile karşılaştırılır:
   - Taranan dosya yolu `map` içinde bulunamıyorsa, bu yeni bir dosya eklendiği anlamına gelir. Gözlemcilere haber verilir ve dosya `map`'e eklenir.
   - Dosya yolu mevcut ancak son değiştirilme tarihi eskisinden farklıysa, dosya üzerinde oynama yapıldığı anlaşılır. `onNotify` tetiklenerek log üretilir ve tarih güncellenir.

IV. GELİŞTİRME SÜRECİNDE KARŞILAŞILAN SORUNLAR VE DÜZELTMELER

Projenin kodlama ve derleme aşamalarında Visual Studio IDE'sinden ve C++ derleyici yapısından kaynaklanan iki büyük pratik sorunla karşılaşılmış ve şu adımlarla çözülmüştür:

A. Derleyici Dil Standardı Uyuşmazlığı ve MSB4086 Hatası

Visual Studio varsayılan proje ayarlarında eski C++ standartlarını baz aldığından, kodun en başındaki `std::filesystem` ad alanı derleyici tarafından tanınmamış ve `E0135` (ad alanı üyesine sahip değil) hatası alınmıştır. Proje özelliklerinden dil standardı `ISO C++17 Standard (/std:c++17)` olarak değiştirildiğinde ise proje dosyasının arka planındaki yapılandırmanın bozulmasından dolayı sayısal karşılaştırma hatası olan `MSB4086` hatası meydana gelmiştir.

Çözüm: Bu hatayı aşmak için Visual Studio kapatılmış, proje klasöründeki derleme geçmişini tutan geçici `x64` klasörleri tamamen temizlenmiştir. IDE yeniden açılarak proje özellikleri önce "Varsayılan" ayarlara çekilmiş, ardından el ile yazmak yerine doğrudan listeden temiz bir şekilde C++17 standardı seçilerek "Rebuild Solution" yapılmıştır. Böylece derleyicinin takılması önlenmiştir.

B. Forward Declaration ve Bağımlılık Sıralaması Hatası

`Monitor.h` dosyasında sınıfların yazım sırasından ötürü `Error C2065: 'logger': bildirimi yapılmamış tanımlayıcı` hatası alınmıştır. C++ derleyicisinin kodu yukarıdan aşağıya okuma mantığı nedeniyle, `FileMonitor` sınıfı kendisinden sonra tanımlanan `SecurityLogger` yapısını ilk etapta görememiştir.

Çözüm: Kod mimarisi bağımlılık sırasına göre yeniden düzenlenmiştir. En üste soyut arayüz (`IObserver`), onun hemen altına somut gözlemci (`SecurityLogger`) ve en alta ise bu gözlemcileri liste halinde tutacak olan ana `FileMonitor` sınıfı taşınarak derleyicinin tüm sınıfları doğru sırayla tanıması sağlanmıştır.

C. Dizin Bulunamadı Hatası ve Çalışma Zamanı Düzenlemesi

Uygulama ilk çalıştırıldığında konsol ekranında "Hata: Belirtilen dizin bulunamadı!" uyarısı alınmıştır. Bunun sebebinin Windows dosya sistemindeki ters çizgi (`\`) karakterinin C++ dilinde kaçış karakteri olarak yorumlanması ve masaüstü klasör yolunun tam eşleşmemesi olduğu tespit edilmiştir.

Çözüm: `main` fonksiyonu içerisindeki klasör yolu, dizindeki gerçek kullanıcı adı gözetilerek `C:\\Users\\ASUS1\\Desktop\\Ödev\\test` şeklinde çift ters çizgi (`\\`) kullanılarak güncellenmiş ve çalışma zamanı hatası giderilmiştir.

V. SONUÇ

Bu çalışma kapsamında, modern C++ kütüphaneleri ve nesne yönelimli tasarım kalıpları harmanlanarak pratik bir güvenlik aracı üretilmiştir. Tasarımda Observer pattern kullanılması, sistemin esnekliğini maksimuma çıkarmıştır. Uygulama test edilmiş; hedef klasöre yeni bir dosya atıldığında veya mevcut bir metin belgesi değiştirildiğinde milisaniyeler içerisinde konsola doğru logların düştüğü gözlemlenmiştir. Proje, bir yazılım geliştiricinin karşılaşabileceği derleyici yapılandırma ve IDE senkronizasyon sorunlarına karşı iyi bir pratik deneyim sunmuştur.

KAYNAKÇA

[1] B. Stroustrup, *The C++ Programming Language*, 4. Baskı. Addison-Wesley, 2013. *(C++ dilinin ve standart kütüphanesinin temel felsefesi için ana kaynak)*

[2] E. Gamma, R. Helm, R. Johnson ve J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. *(Çalışmada kullanılan Observer tasarım deseninin yapısal standartları için referans)*

[3] ISO/IEC, "Information technology — Programming languages — C++", ISO/IEC Standart 14882:2017, 2017. *(Projede aktif olarak kullanılan `std::filesystem` kütüphanesinin C++17 resmi standart dökümantasyonu)*

[4] Microsoft Learn, "Dizin izleme ve dosya sistemi işlemleri kılavuzu," Microsoft, 2024. [Çevrimiçi]. Erişilebilir: https://learn.microsoft.com/en-us/cpp/standard-library/filesystem *(Geliştirme sürecinde karşılaşılan derleyici dili standardı ve MSB4086 yapılandırma hatalarının çözümünde başvurulan resmi IDE dökümanı)*

[5] S. Meyers, *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. O'Reilly Media, 2014. *(Kod mimarisinde kullanılan `std::vector`, akıllı işaretçiler ve modern bellek yönetimi ilkeleri için referans)*